

Лабораторная работа 5 **ГРАФИЧЕСКОЕ ПРЕДСТАВЛЕНИЕ ДАННЫХ, ФОРМИРОВАНИЕ ПРОСТЫХ ОТЧЕТОВ**

Цель работы: ознакомиться с аспектами формирования статистической выборки и построение графиков с использованием компонентов **TDBChart** и **TChartSeries**; ознакомиться с функциональными возможностями генератора отчетов **FastReport**, порядком разработки, предварительного просмотра и печати отчетов; разработать приложение, включающее формирование статистической выборки данных с построением графиков и генерацией отчетов.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с теоретическими сведениями.
2. Сформировать **SQL**-запросы необходимые для построения графиков и создать НД на основе компонентов **ADOQuery**.
3. Организовать настройку и отображения простых графиков
4. Настроить график с предварительной фильтрации данных.
5. При необходимости установить набор компонентов генератора отчетов **FastReport**.
6. Сформировать отчеты и настроить приложение для визуализации отчетов.
7. Ответить на контрольные вопросы.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

В данной лабораторной работе будут рассмотрены аспекты формирования статистической выборки и построение графиков с использованием компонентов **DBChart** и **ChartSeries**. Также познакомимся с функциональными возможностями генератора отчетов **FastReport**, порядком разработки, предварительного просмотра и печати отчетов.

При этом данные используемые для построения графиков и отчетов, как правило, формируются с применением **SQL**-запросов.

5.1 ФОРМИРОВАНИЕ SQL ЗАПРОСОВ ДЛЯ ПОСТРОЕНИЯ ГРАФИКОВ

Для построения графиков воспользуемся информацией о средней стоимости товаров, данными о количестве товаров на складе, а также сведениями о продажах.

Чтобы получить среднюю стоимость товаров в каждой категории будет использован SQL запрос, в котором используется представление **view_goods_catalog**:

```
SELECT
    view_goods_catalog.tgName,
    AVG(view_goods_catalog.gcCost) AS avgCost
FROM view_goods_catalog
GROUP BY view_goods_catalog.tgName
```

Для использования данных этого запроса при построении графика, на **DataModule2** добавим компонент **TADOQuery** (назовем его – **Q_avg_cost_by_type**), и в свойство **SQL** пропишите данный код.

Чтобы получить информацию о продажах по определенным датам необходимо сформировать запрос:

```
SELECT
    view_goods_operations.oDateTime,
    view_goods_operations.oFullCost
FROM view_goods_operations
WHERE view_goods_operations.oIsSale = 1
```

Однако при выполнении данного запроса продажи оформленные в один день будут учитываться как разные позиции (Рис. 5.1, а), поскольку в базе данных хранится информации о дате и времени операции.

Для того чтобы исправить данную ситуацию нам необходимо просуммировать стоимость каждой продажи и сгруппировать результат по дням (Рис. 5.1, б). Изменим запрос следующим образом:

```
SELECT
    CAST(view_goods_operations.oDateTime AS date) AS oSaleDate,
    SUM(view_goods_operations.oFullCost) AS oDayCost
FROM view_goods_operations
WHERE view_goods_operations.oIsSale = 1
GROUP BY oSaleDate
```

oDateTime DATETIME	oFullCost DECIMAL(42, 2)
06.05.2021 11:25:00	86295,00
06.05.2021 14:27:00	129287,00
12.05.2021 12:31:54	25499,00
14.05.2021 10:34:23	27955,00
14.05.2021 12:32:51	129980,00
18.05.2021 14:38:45	25499,00
18.05.2021 12:34:17	49999,00
20.05.2021 11:23:41	25499,00
22.05.2021 15:35:16	64990,00

oSaleDate DATE	oDayCost DECIMAL(64, 2)
06.05.2021	215582,00
12.05.2021	25499,00
14.05.2021	157935,00
18.05.2021	75498,00
20.05.2021	25499,00
22.05.2021	64990,00

а

б

Рис. 5.1. Данные по продажам товаров: а – вывод всех данных;
б – с группировкой по дате

В данном запросе функция **CAST** преобразует значение из одного типа данных в другой. Синтаксис MySQL функции **CAST**:

CAST(значение AS тип_данных)

В функции указывается (передается) значение для преобразования в другой тип данных. В качестве типа данных может быть один из следующих вариантов (таблица 5.1)

Таблица 5.1

Тип данных для преобразования в СУБД MySQL

Тип данных	Описание
DATE	Преобразует значение в тип DATE , который имеет только часть даты. Формат: 'YYYY-MM-DD'. Поддерживаемый диапазон: от '1000-01-01' до '9999-12-31'.
DATETIME	Преобразует значение в тип DATETIME , который имеет как часть даты, так и время. Формат: 'YYYY-MM-DD HH:MM:SS'. Поддерживаемый диапазон: '1000-01-01 00:00:00' до '9999-12-31 23:59:59'.
TIME	Преобразует значение в тип TIME , который имеет только часть времени. Формат: 'HH:MM:SS'. Поддерживаемый диапазон от '-838:59:59' до '838:59:59'.
CHAR	Преобразует значение в тип CHAR , который представляет собой строку фиксированной длины.
SIGNED	Преобразует значение в тип SIGNED , который является 64-разрядным целым числом со знаком.
UNSIGNED	Преобразует значение в тип UNSIGNED , который представляет собой 64-разрядное целое число без знака.
BINARY	Преобразует значение в тип BINARY , который представляет собой двоичную строку.

Измененный запрос пропишите в новом компоненте **ADOQuery**, который назовите **Q_sales_by_days**.

5.2 ДОБАВЛЕНИЕ ГРАФИКОВ В ПРОЕКТ

На графике может одновременно располагаться несколько серий. В большинстве случаев их значения строятся по одинаковому закону, при этом две и более серий показываются на графике одновременно для сравнения. Рассмотрим способы добавления серии на график и настройку ее параметров на уровне программного кода.

Рассмотрим примеры создания графиков, которые выводят информацию о стоимости товара и сведения о продажах.

Сначала создадим новую форму (**Form6**), и подключим к ней **Unit2 (DataModule2)** через меню **File** при помощи пункта **Use Unit** На **Form6** разместим компонент **TPageControl** в котором создадим три вкладки.

Для начала создадим график, отображающий среднюю стоимость товаров в зависимости от типа. Для этого на первой вкладке разместим компонент **TDBChart** (по умолчанию имя **DBChart1**). Войдем в редактор графика (рис 5.2) используя двойное нажатие по компоненту **TDBChart** или пункт **Edit Chart...** контекстного меню.

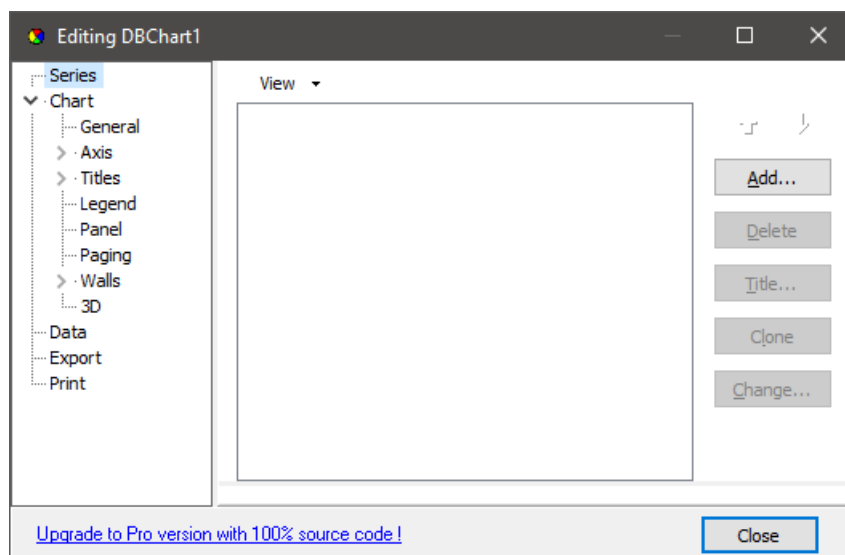


Рис. 5.2. Окно редактора графика

Для добавления серии в график на закладке **Series** нажмем кнопку **Add**, появится окно выбора типа серии. В этом окне представлены базовые типы серий и возможности настройки стиля их отображения. В левом нижнем углу можно увидеть стрелку, при нажатии на которую отобразится галерея всех видов графиков с отображением подтипов (рис. 5.3).

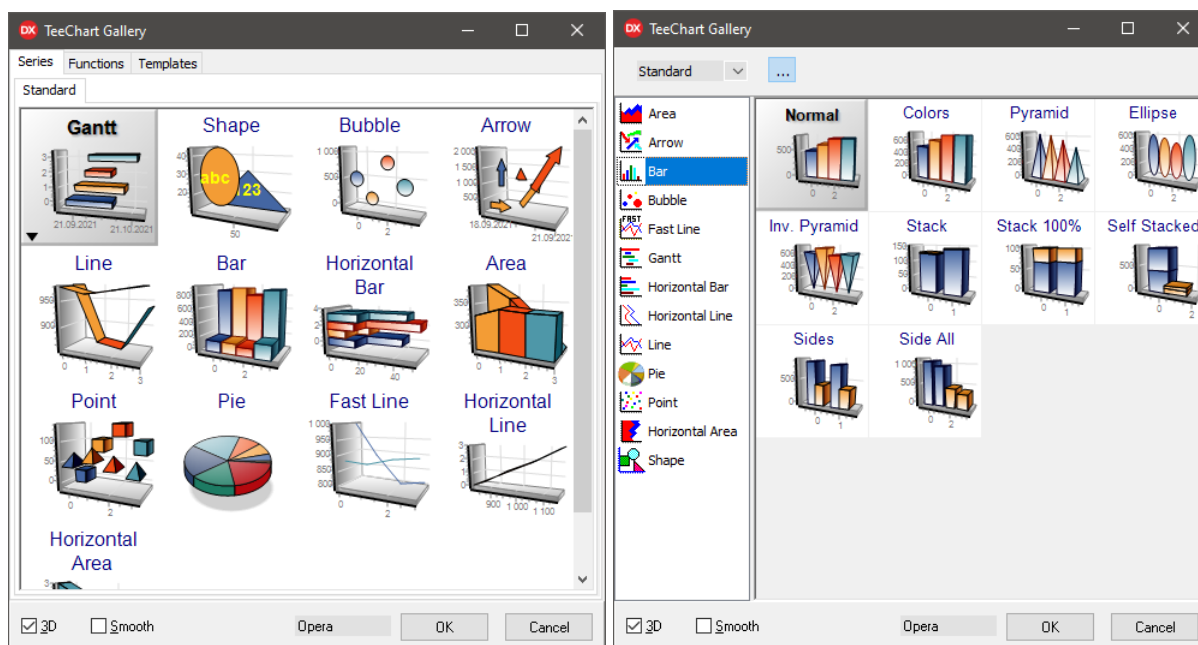


Рис. 5.3. Окно выбора типа серии

Следует отметить, что не все типы серий требуют значений по осям **Y** или **X**. Так, для серий типа **Pie** и **Bar** можно указывать значения по одной из осей и значения меток **Labels**, в качестве которых могут использоваться символьные поля и поля типа даты и времени.

Выберем серию типа **Bar** и нажмем **OK**. В списке серий будет показана серия типа **Bar**, а в дереве параметров будет отображено ее название. При выборе в нем созданной серии **Series1** будет отображен блокнот с вкладками **Format**, **Stack**, **General**, **Marks**, **DataSource** (рис. 5.4).

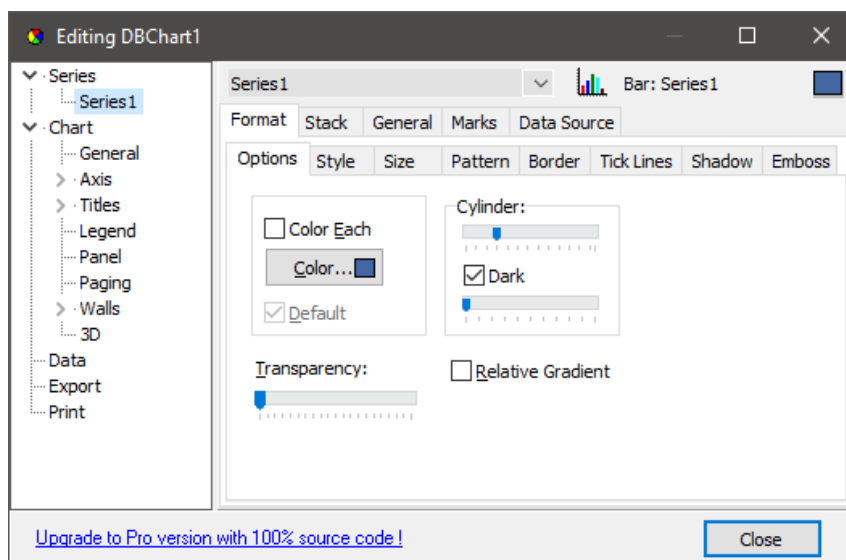


Рис. 5.4. Окно настройки серии

Свойства серии, которые можно установить на страницах, соответствующих этим вкладкам:

- **Format** – отвечает за настройку отображения серии. Здесь можно определять свойства палитры, линий графика и т. д.;
- **Stack** – предоставляет доступ к настройкам вида сбора графика при наличии нескольких серий;
- **General** – позволяет задавать форматы данных;
- **Marks** – устанавливает параметры маркеров (значения в рамке над точками серий). На графике маркеры могут показывать координаты узловых точек. Маркеры отображаются на графике, если отмечено свойство **Visible**. Свойство **Style** дает возможность настроить вид марок;
- **DataSource** – определяет источник данных для серии.

После создания серии необходимо выбрать источник данных для нее. Выпадающий список позволяет определить тип источника данных:

- **Random Values** – набор случайных чисел, который бывает полезен при формировании заготовки серии;
- **Function** – функция, служит для построения графиков на основании данных в двух или более серий;

– **DataSet** – указывает НД, значения полей (столбцов) которого будут использоваться для формирования точек серии.

Из выпадающего списка **DataSet** выберем компонент **Q_avg_cost_by_type**. Укажем, что в качестве меток (**Labels**) будет использоваться поле **tgName**, а поле **avgCost** – значения **Bar**. (рис. 5.5):

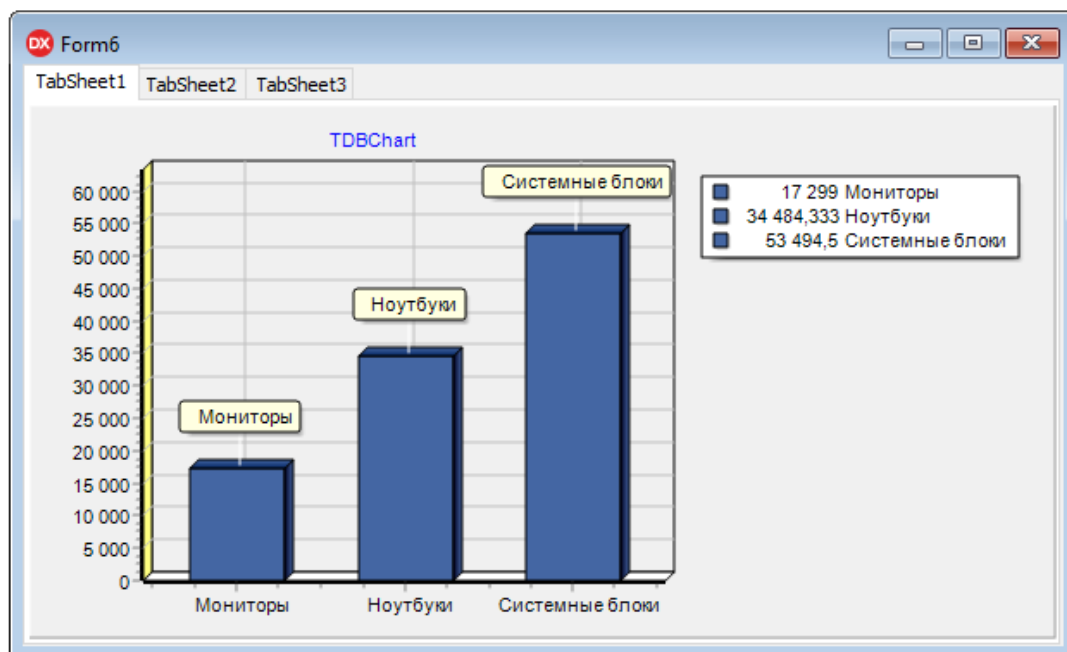
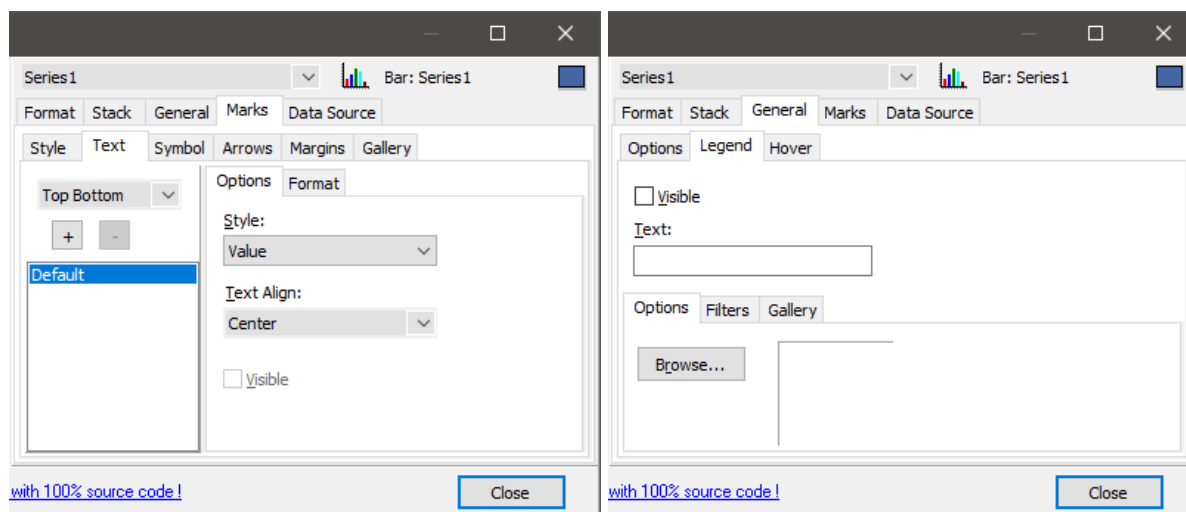


Рис. 5.5. График средней стоимости товаров по категориям

Для данного графика не целесообразно выводить легенду и в маркерах выводить название типа товаров. Для отключения легенды в настройках серии у легенды уберем флажок **Visible** (рис 5.6, а) А выводимый текст в маркерах изменим на конкретное значение (рис 5.6, б)



а

б

Рис. 5.6. Пример настройки графика с изменением отображаемых данных:
а – настройка легенды; б – настройка маркеров.

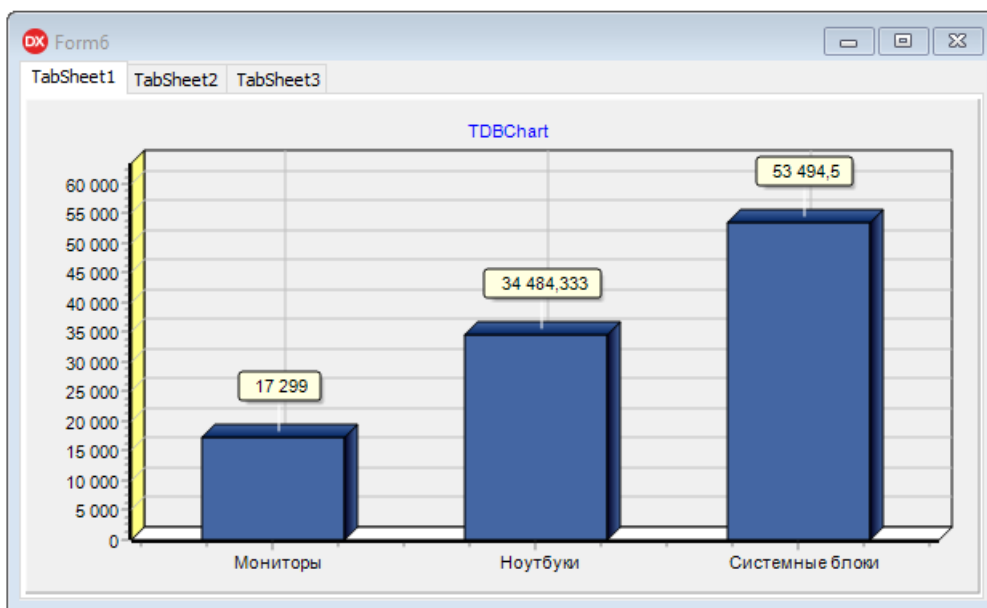


Рис. 5.7. Пример графика с настроенным отображением

Формирование графика для вывода сведений о продажах из НД **Q_sales_by_days** будет происходить схожим образом. Разместим на второй вкладке компонент **TDBChart** (имя по умолчанию **DBChart2**). Вызовем редактор, и на этот раз при создании серии вместо типа **Bar** укажем **Line**.

Настройка источника данных будет следующей: из выпадающего списка **DataSet** выберем компонент **Q_sales_by_days**. Укажем, что поле **oSaleDate** содержит значения по оси **X**, а поле **oDayCost** – значения по оси **Y**. Поскольку поле **oSaleDate** является полем типа **TDateField**, то напротив настройки оси **X** необходимо поставить флажок **DateTime** (рис 5.8).

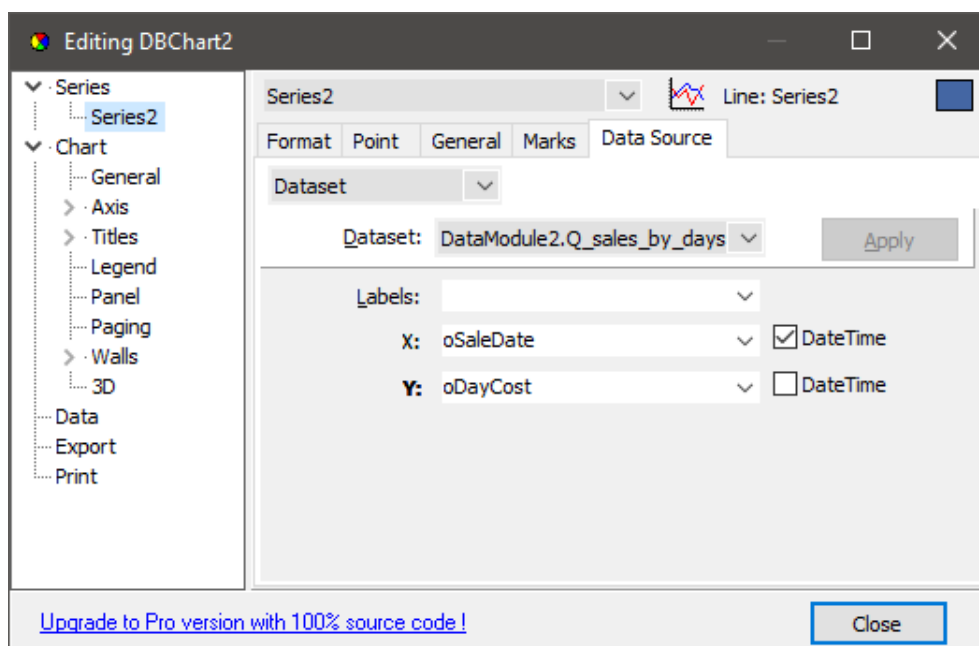
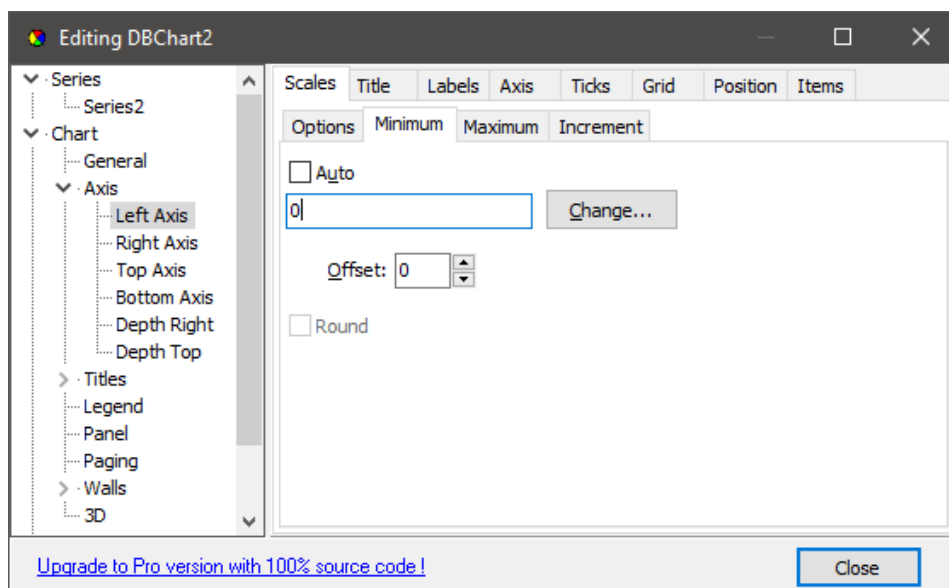
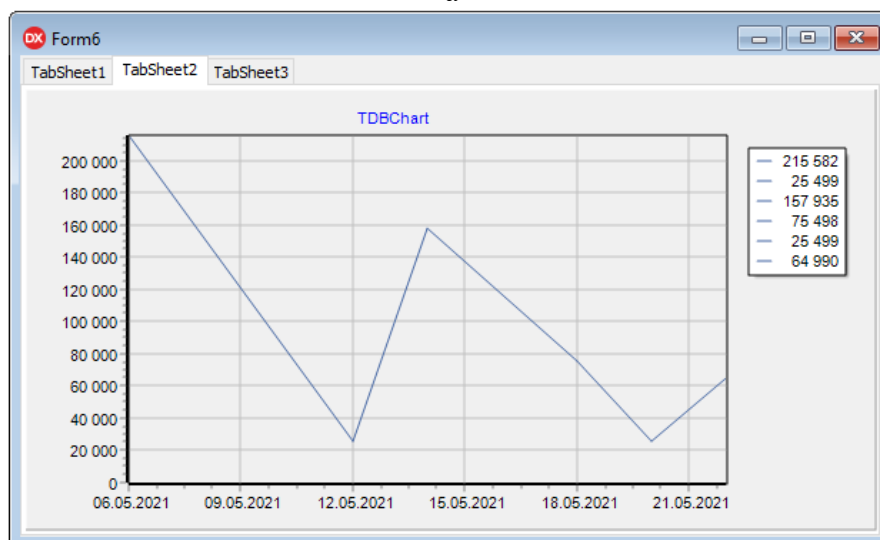


Рис. 5.8. Настройка источника данных графика продаж по датам.

Для того чтобы вертикальная ось не выбирала в качестве начального значения минимальное из возможных, то в ручном режиме установим значение **0** (рис 5.9, а).



а



б

Рис. 5.8. График продаж по датам: а – настройка левой вертикальной оси; б – отображение графика.

Для вызова формы с графиками модифицируем основное меню на главной форме, добавив пункт **Аналитические данные** в котором создадим подпункт **Графики**. Для вызова формы с графиками в обработчике **OnClick** этого пункта пропишем:

```
procedure TForm1.N6Click(Sender: TObject);
begin
    Form6.ShowModal;
end;
```

Схожим образом осуществляется формирование всех видов графиков. При выводе графиков их можно обновлять, изменяя исходный набор данных. Рассмотрим данную возможность на примере отображения складских запасов с возможностью фильтрации по типу товара. Для этого на **DataModule2** добавим компонент **TADOQuery** (назовем **Q_storage_count**) в нем будет **SQL**-код выбора всех данных из **view_storage**.

На **Form6** в третьей вкладке разместим следующие компоненты: **TPanel** и **TDBChart**. Поскольку товаров может быть много, то целесообразно сделать график типа **Horizontal Bar** (рис 5.10). На панель поместим **TDBLookupComboBox** для выбора типа товара, **TButton** для сброса фильтра и **TLabel** для пояснения (рис 5.11).

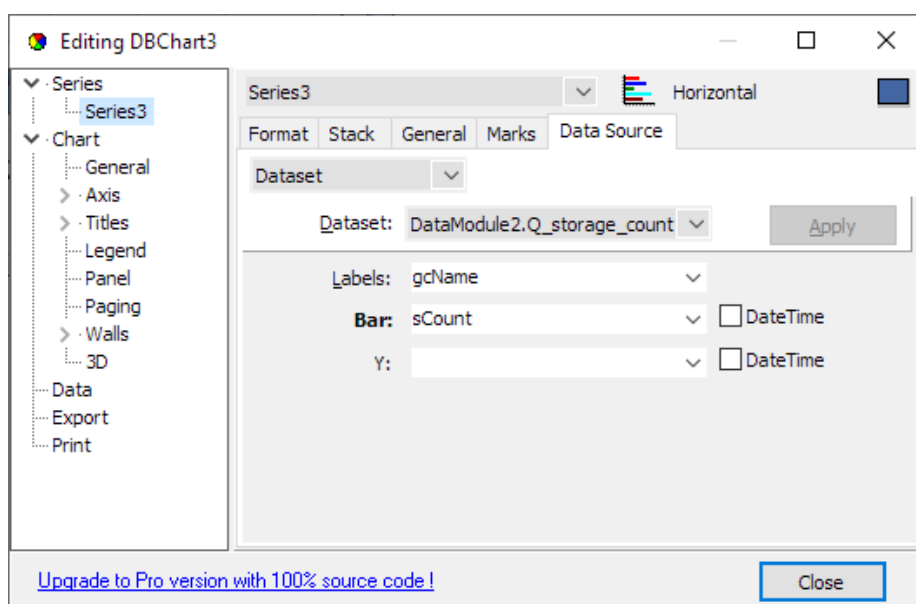


Рис. 5.10. Настройка источника данных для графика.

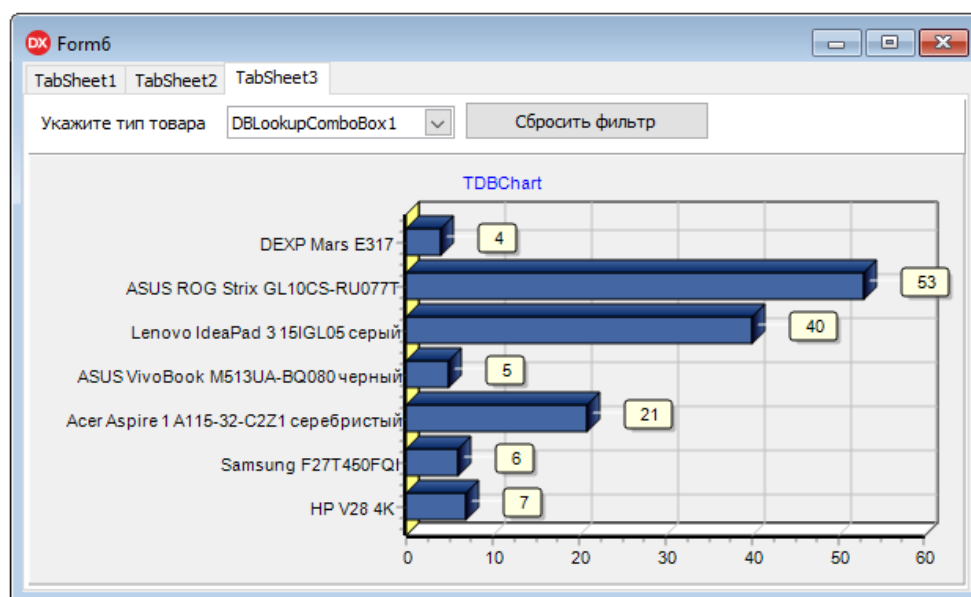


Рис. 5.11. Настройка вкладки с графиком о количестве товара на складе.

Компонент **DBLookupComboBox1** настроим следующим образом (таблица 5.2)

Таблица 5.2

Свойства компонента **DBLookupComboBox1** для выбора типа товаров

Свойство	Значение
ListSource	DataModule2.DS_T_type_goods
ListField	tgName
KeyField	IDtg

Непосредственная организация работы с графиком будет осуществляться с использованием события **OnCloseUp** компонента **DBLookupComboBox1**. Пропишем следующий код в обработчик события:

```
procedure TForm6.DBLookupComboBox1CloseUp(Sender: TObject);
var
    baseSQL, filter_s: string;
begin
    baseSQL := 'SELECT * FROM view_storage ';
    filter_s := '';

    if DBLookupComboBox1.KeyValue <> null then
        filter_s := 'WHERE IDtg = ' + inttostr(DBLookupComboBox1.KeyValue);

    DataModule2.Q_storage_count.sql.text := baseSQL + filter_s;
    DataModule2.Q_storage_count.Active := true;
    DBChart3.RefreshData;
end;
```

Фильтрация данных происходит типичным образом. Обновление данных (перерисовка графика) происходит с применением метода **RefreshData** у компонента **TDBChart**.

Поскольку компонент **TDBLookupComboBox** не предусматривает выбор пустого значения, то для сброса фильтра будем использовать событие **OnClick** кнопки **Сбросить фильтр**:

```
procedure TForm6.Button1Click(Sender: TObject);
begin
    // назначаем пустое значение в качестве выбора
    DBLookupComboBox1.KeyValue := null;
    //имитируем выполнение события OnCloseUp в компоненте DBLookupCombo
    Box1
    DBLookupComboBox1CloseUp(nil);
end;
```

Запустив приложение можно проверить, как будет работать фильтрация данных для графика. Пример работы фильтра продемонстрирован на рис 5.12. Фильтр можно применять к любым данным используемым для отображения на графике (так можно отобразить данные с учетом временного периода).

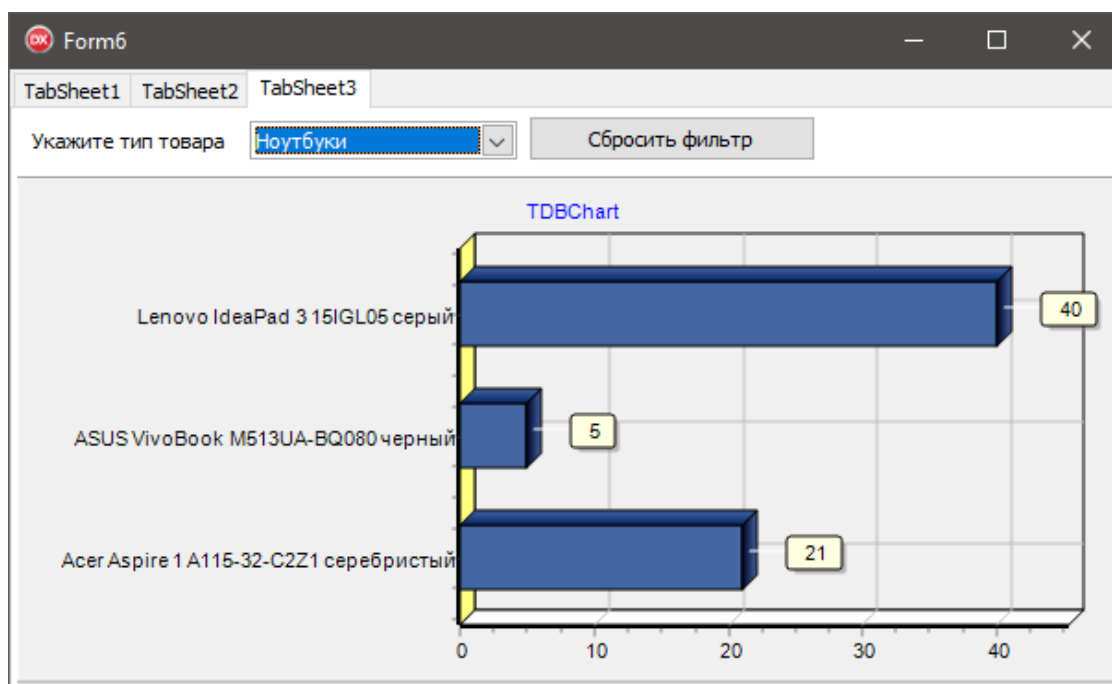


Рис. 5.12. График с выбором категории товаров

Далее необходимо настроить подписи к осям и задать название самому графику с использованием параметра **Title**. Пример настройки заголовка графика на рисунке 5.13, настройка заголовка вертикальной левой оси (**Left Axis**) на рисунке 5.14.

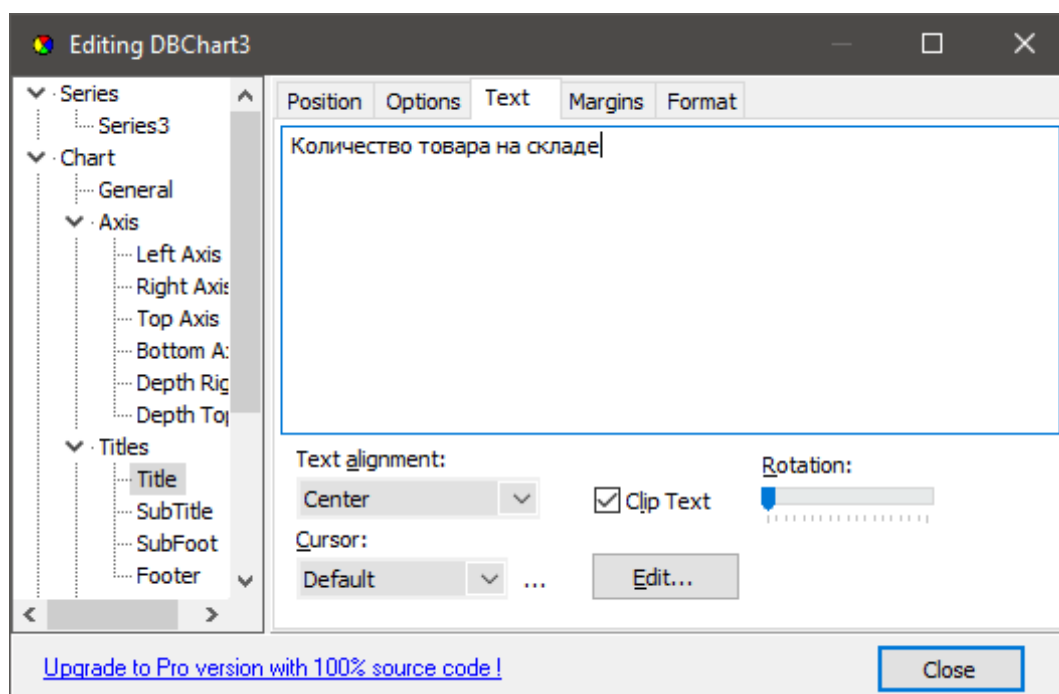


Рис. 5.13. Настройка заголовка графика

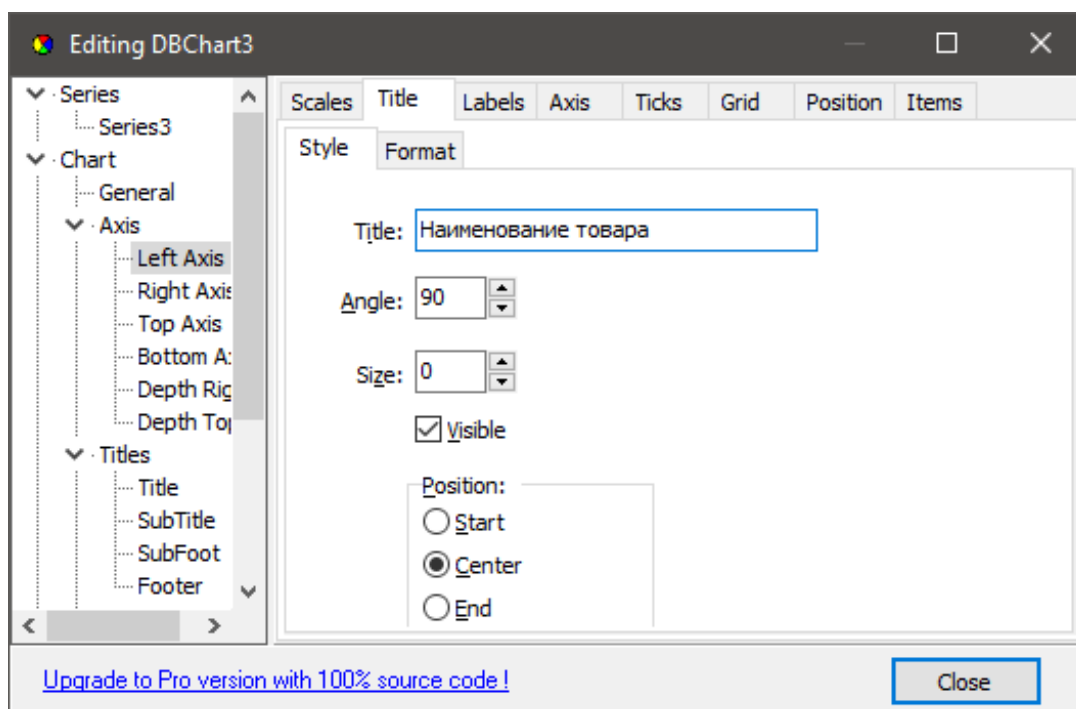


Рис. 5.14. Настройка заголовка левой оси

Подобным образом можно настроить надписи заголовков и других осей. При этом можно задавать позиционирование и угол поворота текста. Пример настроенного графика с активной фильтрацией по другому типу товара показан на рис 5.15.

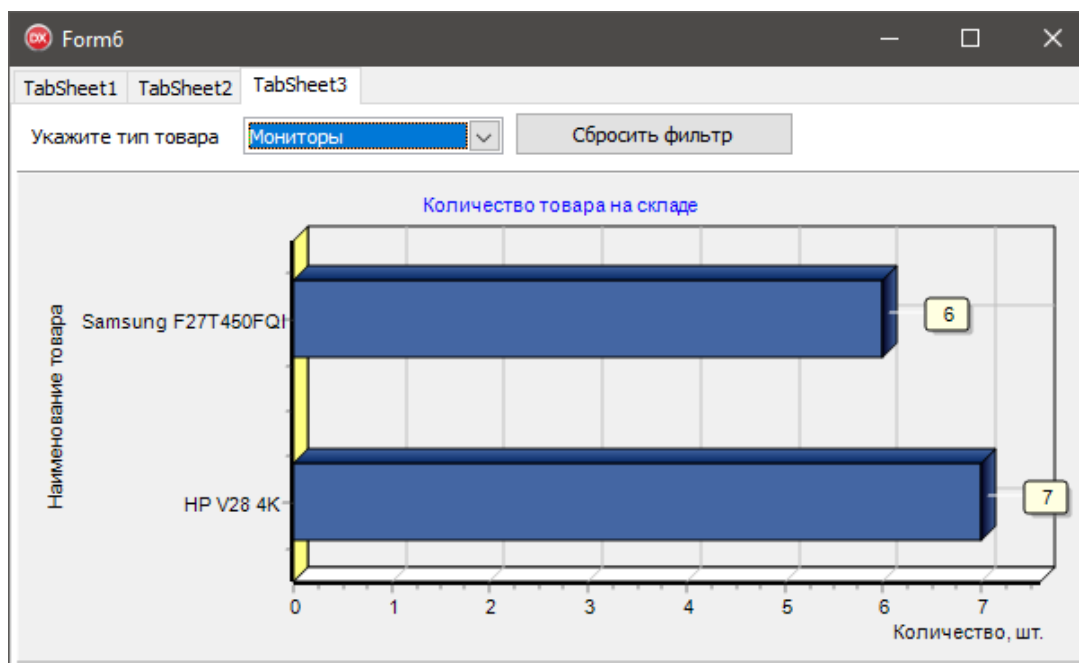


Рис. 5.15. График после настройки осей

Помимо графического представления данных из базы часто используются разнообразные отчеты (сводки информации).

5.3 ИСПОЛЬЗОВАНИЕ ГЕНЕРАТОРА ОТЧЕТОВ

Генератор отчетов – это особый класс программ, востребованный как программистами, так и корпоративными пользователями. Подобных программ-генераторов создано немало, в рамках лабораторной работы ознакомимся с генератором отчетов **FastReport**. Его можно бесплатно скачать с сайта <https://www.fastreport.ru/ru/fastreport-embarcadero-edition/>. Доступен инсталлятор специальной бесплатной версии для Embarcadero RAD Studio, в которой имеется ряд ограничений.

FastReport представлен в виде набора компонентов и дизайнера, инструментарий которого включает огромное количество средств для визуального оформления, создания, настройки и изменения шаблонов отчетов.

В генераторе отчетов **FastReport** пустой отчет представлен в виде листа бумаги. На любом месте этого листа можно расположить объекты, которые могут отображать разную информацию (текст, графика) и определять внешний вид отчета (табл. 5.3).

Таблица 5.3

Объекты дизайнера отчета, входящие в бесплатную версию для Delphi

Объект	Иконка	Описание
Band (бэнд)		Позволяет задать область отчета с определенным поведением
Chart (диаграмма)		Отображает данные в виде диаграмм различных типов (круговой диаграммы, гистограммы и т. п.)
CheckBox		Отображает значок (галочку или крестик)
Text (текст)		Отображает одну или несколько строк текста внутри прямоугольной области
Picture (рисунок)		Отображает графический файл форматов BMP, JPEG, ICO, WMF, EMF
Subreport (вложенный отчет)		Позволяет вставить дополнительный отчет внутрь основного
System text (служебный текст)		Отображает служебную информацию (дату, время, номер страницы), а также агрегатные значения
RichText		Отображает форматированный текст в формате Rich Text (RTF)
OLE		Может отображать любой объект, используя технологию OLE
Gradient (градиент)		Отображает градиентную заливку
Draw (фигура)		Объекты категории Рисование представляют собой различные геометрические фигуры: линии, прямоугольник, прямоугольник с круглыми углами, эллипс, треугольник, ромб
Barcode (штрихкод)		Отображает данные в виде штрихкода. Доступно около десятка разных типов штрихкодов

Основные объекты, с которыми нам придется работать чаще всего, – это объекты **Text** и **Band**. Рассмотрим их более подробно.

Объект Text и его особенности

Объект **Text** обладает очень широкими возможностями. Он может отображать текст, рамку, заливку. При этом текст может выводиться любым шрифтом любого начертания, размера, цвета и стиля.

Все настройки объекта **Text** задаются визуально с помощью панелей инструментов (рис. 5.16).



Рис. 5.16. Панели инструментов настройки свойств объекта *Text*

Познакомимся с некоторыми возможностями объекта **Text** на примере нового объекта, состоящего из двух строк:

Это очень, очень, очень длинная строка текста.

А это вторая строка, покороче.

Включим рамку у объекта и с помощью мыши растянем его до размеров 9×3 см. На макете отчета мы увидим, что объект умеет показывать не только однострочный текст, но и несколько строк. Теперь уменьшим ширину объекта до 5 см и на макете увидим, что длинные строки не уместились в объекте и были перенесены по словам. Сработало свойство объекта **WordWrap** (перенос по словам). Если отключить его в инспекторе объектов или через контекстное меню, то длинные строки будут обрезаны.

Рассмотрим типы выравнивания текста внутри объекта (рис. 5.17, а). Кнопки выравнивания, расположенные на панели инструментов **Текст**, позволяют задавать независимое выравнивание текста по горизонтали и по вертикали. Обратим внимание на кнопку **Выравнивание по ширине**, позволяющее выравнивать текст по обоим краям объекта, при этом должно быть активно свойство **WordWrap**.

Весь текст также может быть повернут на любой угол в пределах от 0 до 360°. Кнопка  на панели инструментов **Текст** позволяет быстро повернуть его на 45, 90, 180 и 270°. Для поворота текста на произвольное значение, нужно воспользоваться свойством **Rotation** в инспекторе объектов, задав нужный угол. При повороте на значения, отличные от 90, 180, 270°, текст может выйти за пределы объекта (рис. 8.3, б). Избавиться от этого недостатка можно за счет небольшого увеличения высоты объекта.

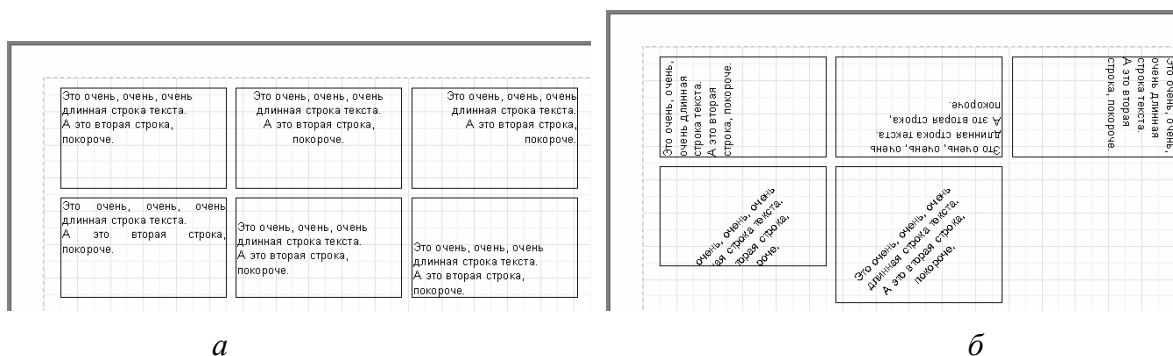


Рис. 5.17. Примеры настройки отображения текста: *а* – выравнивание; *б* – поворот

Среди оставшихся свойств объекта **Text**, которые влияют на его внешний вид, можно выделить следующие:

- **BrushStyle** – тип заливки объекта;
- **CharSpacing** – расстояние между символами в пикселах;
- **GapX, GapY** – отступы текста от левой и верхней границ объекта, в пикселах;
- **LineSpacing** – расстояние между строками в пикселах;
- **ParagraphGap** – отступ первой строки параграфа в пикселах.

Большинство из этих свойств доступны только из инспектора объектов.

Объект **Text** способен обрабатывать простейшие **HTML**-теги, которые могут располагаться внутри текста объекта. По умолчанию теги отключены и чтобы их включить, необходимо назначить значение **True** свойству **AllowHTMLTag** (**HTML**-теги в тексте) в инспекторе объектов или пометить пункт в контекстном меню объекта. Поддерживаемые теги:

- **** – жирный текст;
- **<i>** – наклонный текст;
- **<u>** – подчеркнутый текст;
- **<strike>** – зачеркнутый текст;
- **<sub>** – подстрочный текст;
- **<sup>** – надстрочный текст;
- **** – цвет шрифта;
- **<nowrap>** – текст не разрывается при использовании настройки свойства **WordWrap**, а переносится целиком.

Одной из самых главных особенностей объекта **Text** является возможность отображения не только статического текста, но и выражений, причем выражения могут располагаться в объекте вместе с текстом.

Генератор отчетов **FastReport** автоматически распознает имеющиеся в тексте выражения по квадратным скобкам. Объект **Text** может содержать любое количество выражений, смешанных с обычным текстом. В скобки можно заключать и одиночные переменные, и выражения, например **[1 + 2*(3 + 4)]**. В выражениях можно использовать константы, переменные, функции, поля БД.

Рассмотрим простой пример: поместим в объект **Text** следующую строку:

```
Hello, World! Today is [DATE].
```

Если запустить отчет на построение нажав на кнопку **F9** или выбрав в меню **File** подпункт **Preview**, то мы увидим следующее (дата будет отображена текущая, а не та, фраза что в примере):

```
Hello, World! Today is 01.07.2021.
```

Автоматическое распознавание имеющихся в тексте выражений по квадратным скобкам приводит к тому, что если текст содержит квадратные скобки, то он будет воспринят как выражение. Например, если нам нужно вывести текст

```
a[1] := 10
```

то **FastReport** воспримет **[1]** как выражение и выведет на странице отчета следующий текст:

```
a1 := 10
```

что, естественно нас не устроит. Этого можно избежать, отключив свойство **AllowExpressions** (можно в контекстном меню), и тогда все выражения, имеющиеся в тексте, будут проигнорированы.

Иногда требуется, чтобы текст содержал и выражения, и просто текст с квадратными скобками, например:

```
a[1] := [myVar]
```

Отключение выражений позволит вывести квадратные скобки там, где они нужны, но при этом деактивирует и обработку выражений. В этом случае **FastReport** позволяет задать другой набор символов, обозначающих выражение. За это отвечает свойство объекта **ExpressionDelimiters**, которое по умолчанию равно **[,]**. В нашем случае для выражений можно использовать не квадратные, а угловые скобки если **ExpressionDelimiters** нужно установить значение **< , >** (запятая разделяет открывающие и закрывающие символы):

```
a[1] := <myVar>
```

Однако имеется ограничение: нельзя задавать одинаковые открывающие и закрывающие символы, т. е. значение **%,%** работать не будет. При этом можно задать несколько символов, например **<%,%>**. И тогда наш пример будет выглядеть так:

```
a[1] := <%myVar%>
```

Объект **Text** является универсальным: он дает возможность вывести данные с учетом различных вычислений на основе выражений.

Объект *Band*

Слово **Band** (бэнд) по-английски означает «полоска». Бэнды применяются для логической группировки объектов, т. е. для размещения объектов в нужном месте отчета. В зависимости от типа бэнда мы можем расположить объект вверху или внизу страницы, на первой или на последней странице. Существуют следующие виды бэндов, которые определяют функциональную возможность отображения:

- **Page Header** (заголовок страницы | верхний колонтитул страницы) – выводится в самом верху на каждой странице;
- **Page Footer** (нижний колонтитул страницы) – выводится в самом низу на каждой странице;
- **Report Title** (заголовок отчета) – выводится на первой странице отчета вверху, но после бэнда **Page Header**, который регулируется свойством страницы **TitleBeforeHeader** в инспекторе объектов;
- **Report Summary** (итоги отчета) – выводится в самом конце отчета, на свободном месте;
- **Header** (заголовок) – применяется при организации заголовков для отдельных блоков отчета;
- **Footer** (нижний блок | итоги) – отображается после вывода данных;
- **Data Band** (дата-бэнд, бэнды-данные) – это бэнды с именами **Master Data**, **Detail Data**, **Subdetail Data**, **Data 4th Level**, **Data 5th Level**, **Data 6th Level**, которые используются для отображения данных из набора данных. При построении отчета **FastReport** повторит печать бэнда столько раз, сколько записей будет в таблице, на основе которой выполняется построение отчета. При этом если свободное место на странице закончилось, то будут сформированы новые страницы отчета.

Так, разместив объект на бэнде типа **Page Header** (заголовок страницы | верхний колонтитул страницы), мы тем самым покажем **FastReport**, что данный объект нужно вывести на каждой странице готового отчета вверху. Аналогичным образом бэнд **Page Footer** (нижний колонтитул страницы) выводится внизу каждой страницы со всеми лежащими на нем объектами. Дизайнер **FastReport** автоматически размещает бэнды на странице таким образом, чтобы вверху находились бэнды-заголовки, после них – бэнды-данные, и в самом низу – бэнды-итоги.

5.4 СОЗДАНИЕ ПРОСТОГО ОТЧЕТА И ОТЧЕТА «ЭТИКЕТКА»

Рассмотрим пример создания простого отчета **Список товаров** выводящего сведения из каталога товара (содержит перечень товаров с их стоимостью). Для этого разместим в **DataModule2** компонент **TfrxDBDataset**, в свойстве **DataSet** которого укажем **Q_v_goods_catalog**. Также разместим компонент **TfrxReport** – основной компонент **FastReport** для формирования шаблона отчета. Щелкнув два раза на

TfrxReport, откроется дизайнер отчетов, в меню **File** которого выберем пункт **New report** (рис 5.18):

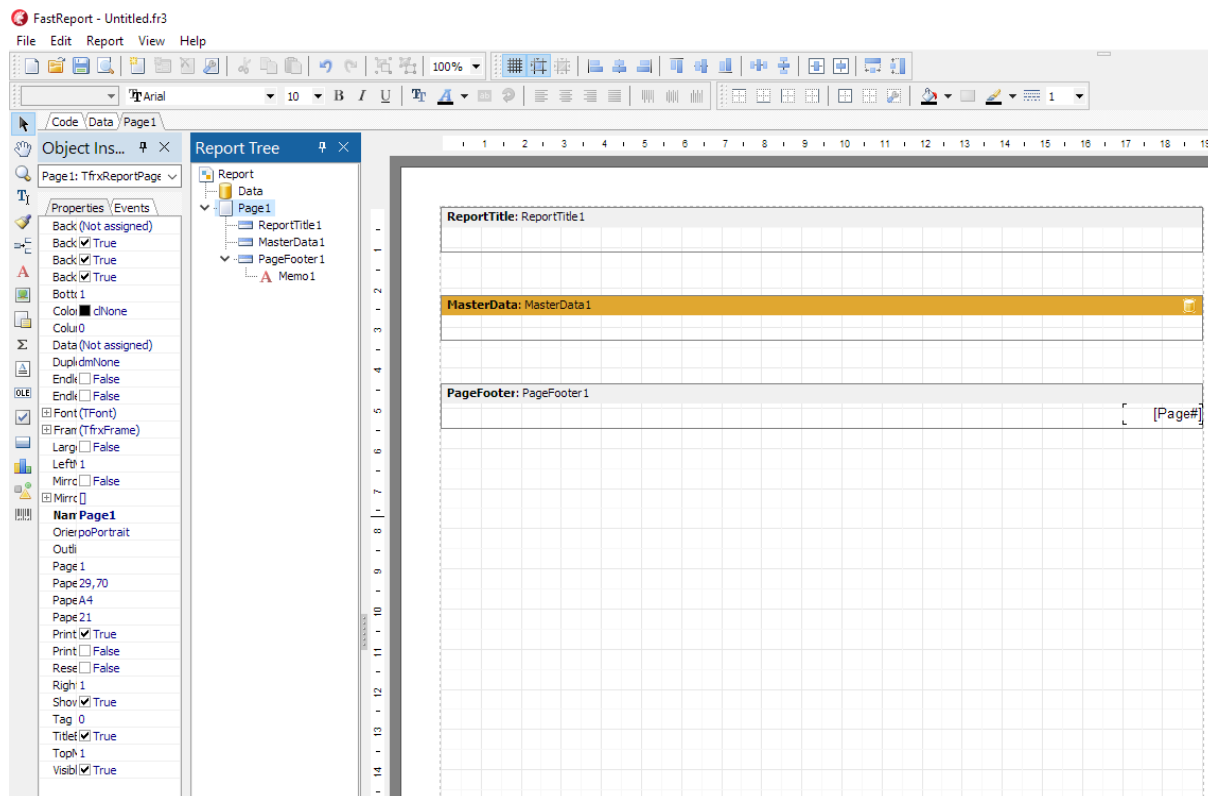


Рис. 5.18. Дизайнер запросов *TfrxReport*

Это было сделано, чтобы **FastReport** автоматически создал пустой шаблон с тремя бэндами: **Report Title**, **Master Data** и **Page Footer**. Чтобы данные запроса были видны в **FastReport**, необходимо разрешить их использование. Для этого выберем пункт меню **Report** затем **Data** и пометим наш набор данных галочкой (рис 5.19):

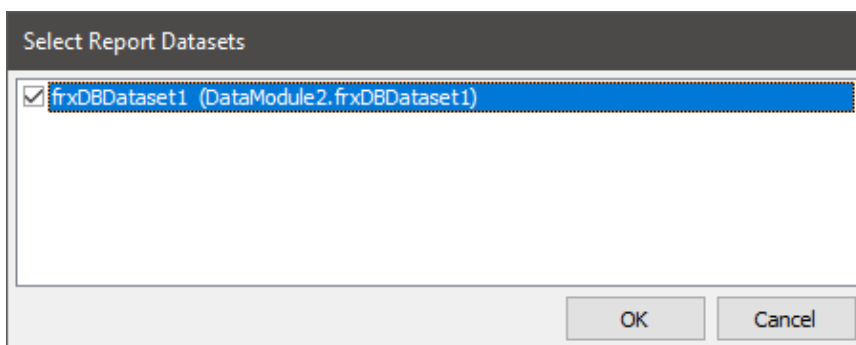


Рис. 5.19. Окно выбора набора данных для отчета

Приступим к созданию отчета. На бэнд **Report Title** поместим объект **Text** с текстом **Список товаров**. Бэнд **Master Data** подключим к нашему источнику данных.

Это можно сделать тремя способами:

- сделать двойной щелчок на бэнде и выбрать НД;
- выбрать пункт **Edit...** из контекстного меню бэнда и указать НД;
- щелкнуть на свойстве **DataSet** в инспекторе объектов и указать НД.

Разместим на бэнде три объекта **Text**, которые будут отображать наименование товара, его тип и стоимость (рис 5.20). Сделаем это наиболее простым способом – использование захвата мышью нужного поля (**drag&drop**) и размещение его в области бэнда.

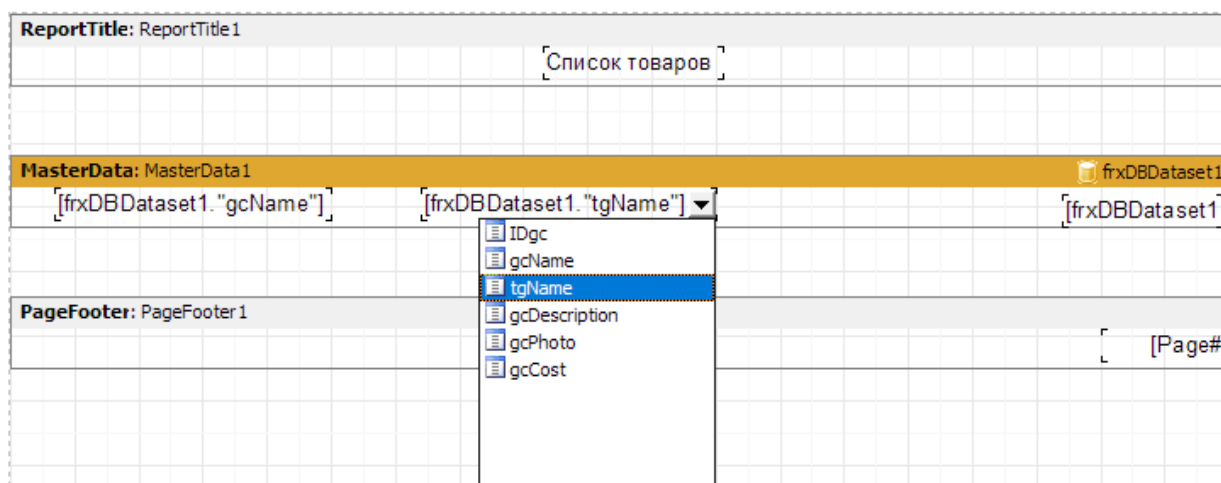


Рис. 5.20. Добавление полей в отчет

После запуска отчета на выполнение в меню **File** подпункт **Preview** или при помощи кнопки **F9** мы увидим сформированный отчет (рис 5.21):

Список товаров		
Acer Aspire 1	Ноутбуки	25499
Lenovo IdeaPad 3	Ноутбуки	27955
ASUS VivoBook	Ноутбуки	49999
DEXP Mars E317	Системные блоки	41999
ASUS ROG Strix	Системные блоки	64990
Samsung F27T450FQI	Мониторы	15199
HP V28 4K	Мониторы	19399

Рис. 5.21. Фрагмент готового простейшего отчета

Далее сформируем отчет в форме **Этикетка** (это отчет, в котором данные о товаре выводятся построчно в небольшой области).

Осуществим создание этикеток к товарам с использованием данных из **Q_v_goods_catalog** (будем использовать уже имеющийся на **DataModule2** компонент **frxDBDataSet1**). Разместим на **DataModule2** еще один компонент **TfrxReport** (имя по умолчанию **frxReport2**), запустим его дизайнер и создадим новый отчет.

Для формирования этикетки товаров необходимо задать в свойствах страницы отчета отображение в виде двух колонок. Это можно сделать в окне настройки параметров страницы (рис 5.22), которое вызывается двойным щелчком мыши по пустой области страницы, либо через пункт **Edit** контекстного меню страницы.

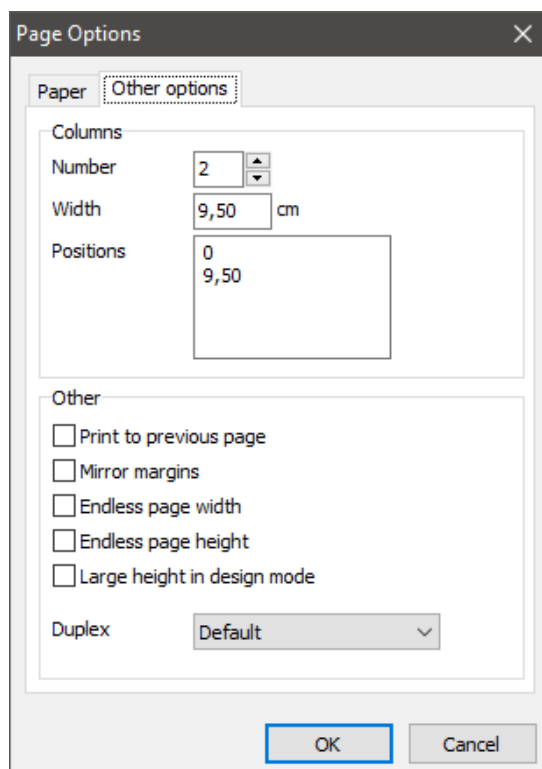


Рис. 5.22. Настройка параметров страницы в две колонки

Разместим элементы и настроим **MasterData** как показано на рисунке 5.23. Поскольку описание имеет довольно длинное содержание, то оно все не помещается при просмотре отчета.

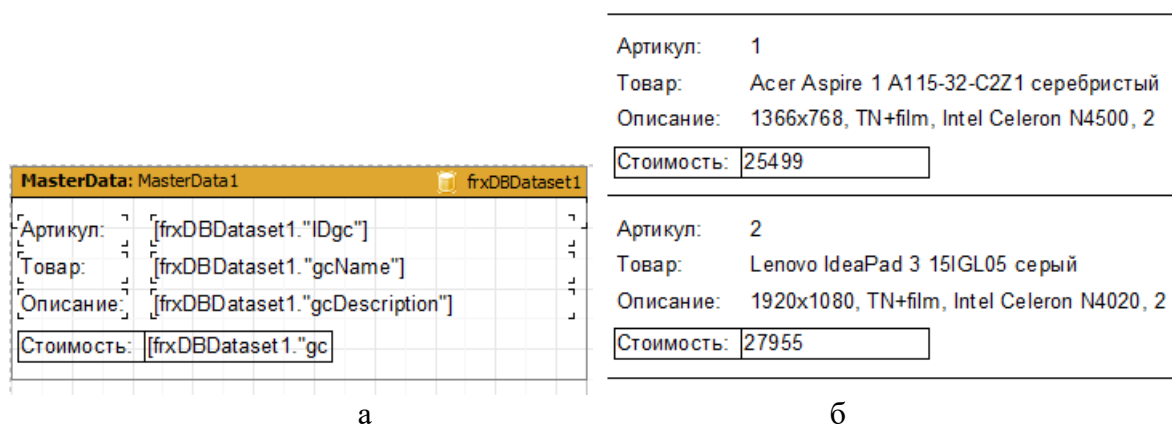


Рис. 5.23. Пример этикетки: а – макет; б – фрагмент готового отчета

Для того чтобы область описания содержала весь доступный текст необходимо задать свойству **StretchMode** объекта **Text**, отображающего поле **frxDBDataset1."gcDescription"**, значение **smActualHeight**.

У бэнда **MasterData1** включим растягивание путем установки свойства **Stretched** в значение **True**. Запустив отчет при предварительном просмотре, мы увидим следующие этикетки (рис. 5.24):

Артикул:	1
Товар:	Acer Aspire 1 A115-32-C2Z1 серебристый
Описание:	1366x768, TN+film, Intel Celeron N4500, 2
Стоимость:	x 1.1 ГГц, RAM 8 ГБ, eMMC 128 ГБ, Intel UHD Graphics
	25499
Артикул:	2
Товар:	Lenovo IdeaPad 3 15IGL05 серый
Описание:	1920x1080, TN+film, Intel Celeron N4020, 2
Стоимость:	x 1.1 ГГц, RAM 8 ГБ, SSD 256 ГБ, Intel UHD 600, Wi-Fi, DOS
	27955

Рис. 5.24. Фрагмент готового отчета с растягиванием

Объект с данными поля **frxDBDataset1."gcDescription"** содержит длинный текст и поэтому он растянулся на несколько строк, а находящийся под ним объект, привязанный к полю также сместился ниже.

Это произошло потому, что в инспекторе объектов по умолчанию все объекты имеют включенное свойство **ShiftMode** (**ShiftMode = smAlways**). Такие объекты смещаются вниз, если над ними есть растягиваемый объект, при этом высота, на которую смещается объект, зависит от того, сколько данных находиться в элементе.

Однако это неприемлемо, необходимо, чтобы объект с текстом **Стоимость** также смещался. Для этого в **FastReport** есть специальный тип бэнда – **Child**-бэнд, который привязывается к основному бэнду и выводится после него. Модифицируем наш отчет как показано рис. 5.25. Для того чтобы связать основной бэнд с дочерним, у бэнда **MasterData1** в инспекторе объектов установим свойство **Child** в значение **Child1**. Теперь каждый раз при печати основного бэнда будет выводиться и дочерний.

Как видим, теперь подписи к соответствующим данным напечатаются там, где нужно. Для того чтобы избежать переноса **Child**-бэнда на следующую страницу, т. е. его отрыва от основного бэнда, в инспекторе объектов у основного бэнда свойство **KeepChild** необходимо установить в значение **True**.

MasterData: MasterData1	
Артикул:	[frxDBDataset1."IDgc"]
Товар:	[frxDBDataset1."gcName"]
Описание:	[frxDBDataset1."gcDescription"]
Child: Child1	
Стоимость:	[frxDBDataset1."gc"]

а

Артикул:	1
Товар:	Acer Aspire 1 A115-32-C2Z1 серебристый
Описание:	1366x768, TN+film, Intel Celeron N4500, 2 x 1.1 ГГц, RAM 8 ГБ, eMMC 128 ГБ, Intel UHD Graphics
Стоимость:	25499
Артикул:	2
Товар:	Lenovo IdeaPad 3 15IGL05 серый
Описание:	1920x1080, TN+film, Intel Celeron N4020, 2 x 1.1 ГГц, RAM 8 ГБ, SSD 256 ГБ, Intel UHD 600, Wi-Fi, DOS
Стоимость:	27955

б

Рис. 5.25. Пример отчета: а – макет; б – фрагмент готового отчета

5.5 СОЗДАНИЕ ОТЧЕТОВ С ИСПОЛЬЗОВАНИЕМ MASTER-DETAIL И ГРУППАМИ

Мы рассмотрели отчеты, в которых присутствовал только один датабэнд – **MasterData** (данные первого уровня). Это давало возможность печатать данные из одной таблицы БД. **FastReport** позволяет печатать отчеты, содержащие до шести уровней данных (в реальных приложениях в основном формируются отчеты с одним-тремя уровнями).

Создадим двухуровневый отчет, отражающий сведения об операциях. Он будет содержать данные из НД **Q_v_goods_operations** и **Q_v_operation_goods_list**: первый – это перечень операций, второй – список товаров, участвующих в операции.

Для создания отчета разместим два компонента **frxDBDataSet** и один компонент **TfrxReport** (с именем **frxReport3**). Теперь настроим компоненты. Один из компонентов **TfrxDBDataSet** назовем **op_master** (изменим свойства **Name** и **UserName**), а другой **op_detail**. В свойстве **DataSet** укажем **Q_v_goods_operations** и **Q_v_operation_goods_list** для **op_master** и **op_detail** соответственно.

Поскольку при реализации лабораторных работ связь *Master-Detail* была реализована, то в текущий момент ничего не меняем в НД. При необходимости нужно организовывать связь *Master-Detail*.

Перейдем к созданию шаблона отчетов. В дизайнера отчета подключим источники данных и разместим на странице макета бэнды **MasterData** и **DetailData**, после чего настроим объекты **Text** для отображения данных из соответствующих полей (рис. 5.26). В **MasterData** будут отображаться данные полей: **op_master."oDateTime"** и **op_master."opType"**.

В **DetailData** будут отображаться данные полей: **op_detail."tgName"**, **op_detail."gcName"**, **op_detail."ocCount"**, **op_detail."gcCost"** и **op_detail."ogcFCost"**.

MasterData: MasterData1					op_master	
Время операции: [op_master."oDateTime"]		Тип операции: [op_master."opType"]				
DetailData: DetailData1					op_detail	
[op_detail."tgName"]	[op_detail."gcName"]		[op_detail."g	[op_detail."o		

Рис. 5.26. Фрагмент макета отчета типа *Master–Detail*

Обратим внимание на то, что бэнд **MasterData** должен располагаться выше бэнда **DetailData**. В противном случае **FastReport** при запуске отчета сообщит об ошибке. Фрагмент готового *Master–Detail* отчета показан на рис 5.27:

Сведения об операциях					
Время операции: 06.05.2021 11:25:00			Тип операции: Продажа		
Ноутбуки	Acer Aspire 1 A115-32-C2Z1 серебристый	1	25499	25499	
Мониторы	Samsung F27T450FQI	4	15199	60796	
Время операции: 06.05.2021 14:27:00			Тип операции: Продажа		
Ноутбуки	Acer Aspire 1 A115-32-C2Z1 серебристый	1	25499	25499	
Системные блоки	ASUS ROG Strix GL10CS-RU077T	1	64990	64990	
Мониторы	HP V28 4K	2	19399	38798	
Время операции: 07.05.2021 11:15:00			Тип операции: Поставка		
Ноутбуки	Lenovo IdeaPad 3 15IGL05 серый	5	27955	139775	
Ноутбуки	ASUS VivoBook M513UA-BQ080 черный	2	49999	99998	
Системные блоки	DEXP Mars E317	4	41999	167996	

Рис. 5.27. Фрагмент отчета типа *Master–Detail*

FastReport позволяет создать аналогичный отчет на основе одного набора данных, сформированного особым способом (использование групп).

Рассмотрим процесс формирования отчета с группами на примере отчета о складских запасах с группировкой по типу товара. Разместим в **DataModule2** компонент **TfrxDBDataset** (**frxDBDataset2**). В свойстве **DataSet** укажем **Q_v_storage**. Добавим на **DataModule2** еще один компонент **TfrxReport** (имя по умолчанию **frxReport4**), запустим его дизайнер и создадим новый отчет. В качестве источника данных выберем **frxDBDataset2** и дополнительно поместим на странице макета бэнд **Group Header**. При размещении бэнда необходимо будет задать параметры для группировки данных (Рис 5.28). В блоке **Break on** укажем условие **Data field**, в котором выберем источник данных **frxDBDataset2** и поле данных **tgName** (для группировки будет использовано поле **tgName**).

Следует обратить внимание что в группу объединяется информация согласно выбранному полю. Если по данному полю не применена сортировка, то тогда может образоваться несколько одинаковых групп.

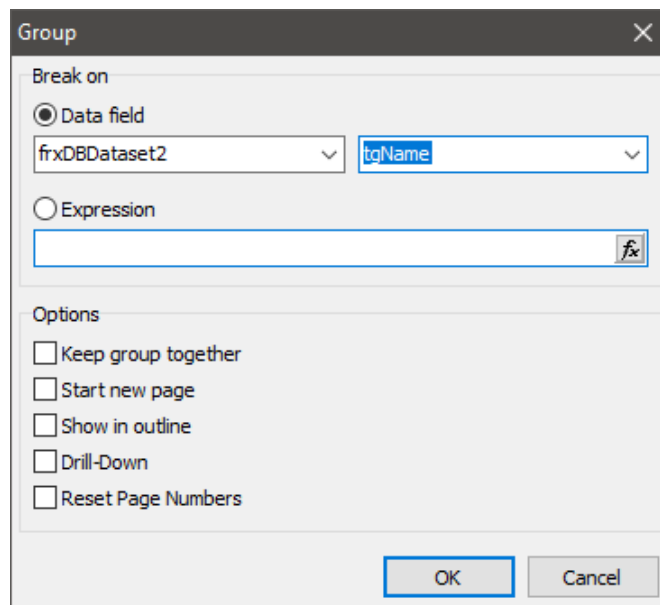


Рис. 5.28. Настройка параметров группировки.

Дата-бэнд **MasterData** привяжем к источнику данных **frxDBDataset2** и разместим объектами следующим образом (рис. 5.29, а). Обратим внимание на то, что заголовок группы должен располагаться над дата-бэндом.

GroupHeader: GroupHeader1	frxDBDataset2."tgName"
[frxDBDataset2."tgName"]	
MasterData: MasterData1	frxDBDataset2
[frxDBDataset2."gcName"]	[frxDBDataset2."tgName"]

а

Мониторы	
HP V28 4K	7
Samsung F27T450FQI	6
Ноутбуки	
Acer Aspire 1 A115-32-C221 серебристый	21
ASUS VivoBook M513UA-BQ080 черный	5
Lenovo IdeaPad 3 15IGL05 серый	40
Системные блоки	
ASUS ROG Strix GL10CS-RU077T	53
DEXP Mars E317	4

б

Рис. 5.29. Пример отчета с группировкой: а – фрагмент макета; б – фрагмент отчета

Аналогичным образом можно строить отчеты с вложенными группами, при этом количество вложений не ограничено. Рассмотрим особенности отчетов с использованием групп.

При печати отчета может возникнуть ситуация, когда часть данных группы будет находиться на одной странице, а часть – на другой. **FastReport** позволяет повторить вывод заголовка группы на следующей странице. Для этого у бэнда **Group Header** нужно включить свойство

ReprintOnNewPage (выводить на новой странице) в инспекторе объектов или контекстном меню. Избежать разрыва группы можно включив свойство заголовка группы **KeepTogether** (держат вместе) в контекстном меню или в инспекторе объектов. При этом если вся группа не помещается на странице, то ее вывод будет перенесен на новую страницу.

При печати группы можно сбрасывать нумерацию страниц, используя свойство **ResetPageNumbers**. При установке данного свойства также следует включить свойство **StartNewPage** (формировать новую страницу), чтобы каждая группа печаталась начиная с новой страницы. Вывести номер страницы и общее число страниц в группе можно с помощью переменных **[Page]**, **[TotalPages]**, помещенных в объект **Text**.

У заголовка группы есть свойство **DrillDown** и аналогичный пункт в контекстном меню. Включение этого свойства позволяет сделать группу интерактивной, т. е. реагирующей на щелчок мыши в окне предварительного просмотра. Щелкая мышью на заголовке группы, ее можно развернуть (показать все ее записи) или свернуть, оставив только заголовок и, при необходимости, нижний блок, который называют подвалом. Показ подвала настраивается свойством **ShowFooterIfDrillDown**.

В большинстве случаев в групповых отчетах необходимо выводить некую итоговую информацию: сумму по группе, количество элементов группы и т. п. В **FastReport** для этих целей существуют агрегатные функции, с помощью которых можно подсчитать функцию от определенного значения по диапазону данных (табл. 5.4).

Таблица 5.4

Список агрегатных функций

Функция	Описание
SUM	Возвращает сумму заданного выражения
MIN	Возвращает минимальное значение заданного выражения
MAX	Возвращает максимальное значение заданного выражения
AVG	Возвращает среднее значение заданного выражения
COUNT	Возвращает количество строк в диапазоне данных

Синтаксис всех агрегатных функций (за исключением функции **COUNT**) следующий (рассмотрим его на примере функции **SUM**):

```
SUM(expression, band, flags)
SUM(expression, band)
SUM(expression)
```

Параметры имеют следующие значения:

- **expression** – выражение для обработки;
- **band** – имя дата-бэнда, по которому будет идти обработка;
- **flags** – битовое поле, которое может содержать следующие значения и их комбинации: **1** – учитывать невидимые бэнды; **2** – накапливать

значение, т. е. не сбрасывать при очередном выводе; **3** – это битовая комбинация **1** и **2**, что означает, что нужно учитывать невидимые бэнды и не сбрасывать сумму.

Обязательным параметром является только параметр **expression**, остальные при вызове функции могут быть опущены. Тем не менее всегда рекомендуется использовать параметр **band**, что позволит избежать ошибок.

Функция **COUNT** имеет следующий синтаксис:

```
COUNT(band, flags)
COUNT(band)
```

Назначение параметров этой функции аналогично вышеописанным.

Существует общее для всех агрегатных функций правило: функция может быть подсчитана только для дата-бэнда и выведена только в бэнде-итоге. К бэндам-итогам относятся **Footer**, **Page Footer**, **Group Footer**, **Column Footer**, **Report Summary**.

Работу агрегатных функций рассмотрим на примере нашего отчета с группами. Добавим в отчет новые элементы (рис. 5.31, а). Поле **frxDBDataset2."sCount"** на дата-бэнде отображает количество конкретного товара на складе. В бэнд **Group Footer** поместим объект **Текст**, содержащий вызов агрегатной функции расчета среднего значения. Он будет отображать общее количество товаров для группы. Пример настройки показан на Рис 5.30. Вставить в нужное место конструкцию агрегатной функции поможет кнопка **Insert Agregate** (со знаком суммы).

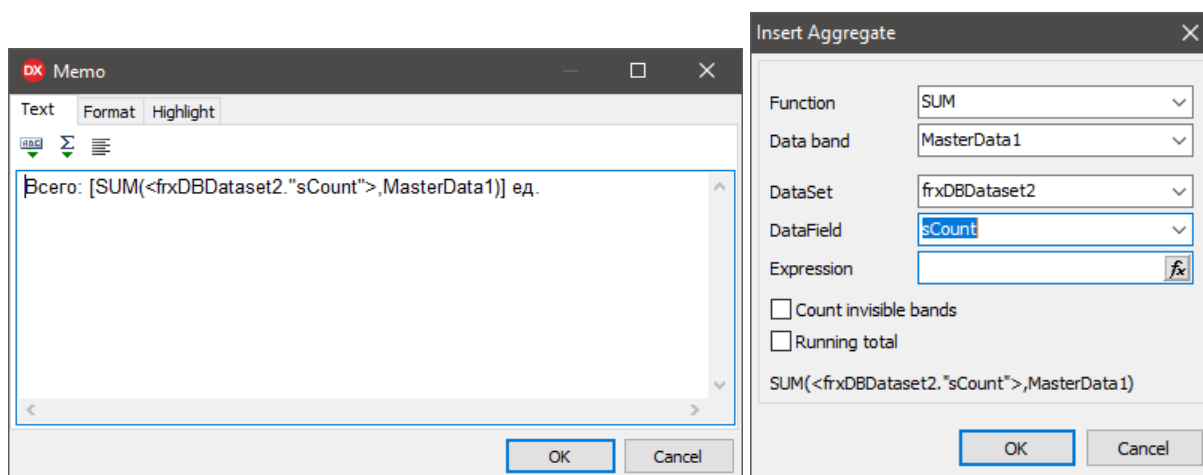


Рис. 5.30. Настройка выводимого сообщения с агрегатной функцией.

Пример фрагмента отчета с группировкой и агрегатной функцией, а также активным свойством **DrillDown** приведен на рисунке 5.31

GroupHeader: GroupHeader1		frxDBDataset2."tgName"
[frxDBDataset2."tgName"]		
MasterData: MasterData1		frxDBDataset2
[frxDBDataset2."gcName"]		[frxDBDataset2."sCount"]
GroupFooter: GroupFooter1		
		Всего: [SUM(<frxDBDataset2."sCount">,MasterData1)] ед.

а

Мониторы	
HP V28 4K	7
Samsung F27T450FQI	6
Всего: 13 ед.	
Ноутбуки	
Системные блоки	
ASUS ROG Strix GL 10CS-RU077T	53
DEXP Mars E317	4
Всего: 57 ед.	

б

Рис. 5.31. Пример отчета с группировкой и агрегатной функцией:
а – фрагмент макета; б – фрагмент отчета с частично развернутыми группами

Настройка формата отображения данных происходит посредством диалогового окна, вызываемого для выделенного объекта через пункт **DisplayFormat** контекстного меню или в инспекторе объектов с помощью редактора свойства **DisplayFormat**.

Особенностью объекта **Text** является то, что он позволяет выделить объект цветом в зависимости от какого-либо условия. В качестве условия может выступать любое выражение.

Рассмотрим выделение на примере складских запасов. Так, у товаров, количество которых менее 6 фон будет выделен красным оттенком. Для этого выберем в **MasterData** объекты **frxDBDataset2."gcName"** и **frxDBDataset2."sCount"**, после чего нажмем кнопку **Highlight** (условное выделение) на панели инструментов дизайнера. Добавим условие **<frxDBDataset2."sCount"> < 6** и настроим оформление – **Style** (Рис. 5.32).

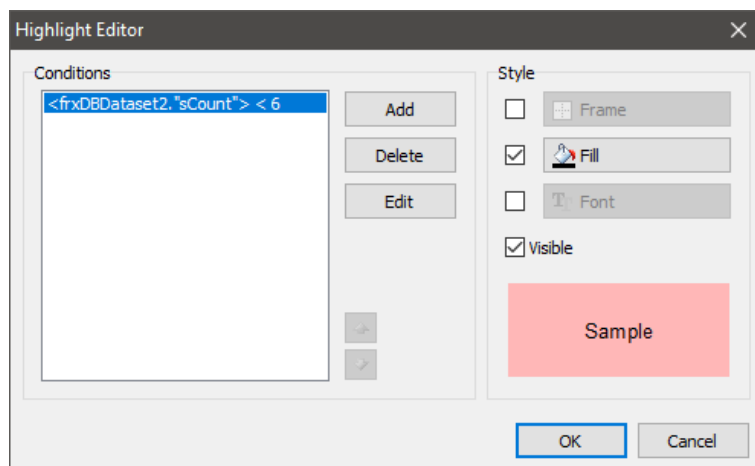


Рис. 5.32. Настройка условного выделения

В окне предварительного просмотра можно увидеть как изменилось отображение отчета (Рис. 5.33).

Мониторы	
HP V28 4K	7
Samsung F27T450FQI	6
Всего: 13 ед.	
Ноутбуки	
Acer Aspire 1 A115-32-C2Z1 серебристый	21
ASUS VivoBook M513UA-BQ080 черный	5
Lenovo IdeaPad 3 15IGL05 серый	40
Всего: 66 ед.	
Системные блоки	
ASUS ROG Strix GL10CS-RU077T	53
DEXP Mars E317	4
Всего: 57 ед.	

Рис. 5.32. Пример отчета с условным выделением

Визуальное оформление условного выделения включают в себя несколько настроек: рамка, шрифт, заливка и видимость объекта. При этом можно включать и выключать каждую настройку для каждого условия.

5.6 ОТОБРАЖЕНИЕ ОТЧЕТА ИЗ ПРИЛОЖЕНИЯ

При программной реализации запуск отчета на выполнение осуществляется методом компонента **TfrxReport**. Метод

procedure ShowReport(ClearLastReport: Boolean = True);

запускает отчет на выполнение и показывает результат в окне предварительного просмотра. Если параметр **ClearLastReport** имеет значение **False**, то отчет добавляется к ранее построенному, в противном случае последний очищается (по умолчанию).

Для того чтобы продемонстрировать возможность показа отчета во время выполнения программы создадим пункты в блоке **Аналитические данные** основного меню. Данные пункты будут соответствовать отчетам: «Отчет "Каталог товаров"», «Отчет "Этикетка"», «Отчет по операциям» и «Отчет о складских запасах». Пропишем обработчики события **OnClick** для этих элементов меню:

```

procedure TForm1.N7Click(Sender: TObject);
begin
    // показ отчета "Каталог товаров"
    DataModule2.frxReport1.ShowReport();
end;

procedure TForm1.N8Click(Sender: TObject);
begin
    // показ отчета "Этикетка"
    DataModule2.frxReport2.ShowReport();

```

```

end;
procedure TForm1.N9Click(Sender: TObject);
begin
    // показ отчета по операциям
    DataModule2.frxReport3.ShowReport();
end;
procedure TForm1.N10Click(Sender: TObject);
begin
    // показ отчета о складских запасах
    DataModule2.frxReport4.ShowReport();
end;

```

Также вызов отчета можно назначить и на обработчик события **OnClick** у кнопки. Стоит помнить, что отчеты выводят данные из набора данных. Соответственно если к НД применить фильтр (осуществить выборку данных) то в отчете будут отображены данные с учетом фильтрации.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Для чего применяется функция **CAST** в **SQL**?
2. Каким образом создается график?
3. Какие виды серий поддерживает компонент **DBChart**?
4. Дайте определение серии графика.
5. Опишите закладки **General**, **Axis**, **Titles** блока настроек **Chart**.
6. За настройку каких параметров отвечают вкладки **Format**, **General**, **Marks** для выбранной серии?
7. Назовите источник выбора данных для серии.
8. Какие координаты указываются для различных типов серии?
9. Охарактеризуйте объект **Text**, какие возможности он предоставляет по настройке отображения.
10. Охарактеризуйте особенности отображение выражений с помощью объекта **Text**.
11. Какие виды объекта **Band** присутствуют в дизайнера отчетов **FastReport**?
12. Назовите этапы создания отчета и включения его в приложение.
13. Охарактеризуйте объект **Child-band** и области его применения.
14. Приведите порядок создания отчета «один-ко-многим».
15. Опишите процесс создания отчета с группировкой.
16. Что представляют собой разворачиваемые (**drill-down**) группы? Как происходит их настройка?
17. Какие агрегатные функции доступны в **FastReport**? Перечислите известные вам способы вставки агрегатных функций.
18. Каким образом осуществляется вывод отчетов на отображение в разрабатываемом приложении?